

High-Level Design Document for LLM Semantic Cache project

Author	Leen Kabaha, Asmaa Aghbaria, Hala Atamna
Version	V0.1
Created	18/5/2026
Last Updated	20/5/2026

1 Introduction

This section specifies the subset of the functional specification that will actually be implemented in the project.

1.1 Overview

This document describes the high-level design of the LLM Semantic Cache project. The project implements a transparent cache layer in front of a remote chat-based Large Language Model (LLM). When a user submits a prompt, the system checks whether a semantically similar prompt already exists in the cache. If a close enough match is found, the cached reply is returned instead of calling the remote LLM. This reduces remote API calls, latency, communication overhead, compute cost, and energy consumption for repeated or near-duplicate queries.

The system is designed as cooperating processes with clear REST interfaces: a CLI client, a cache orchestrator, an embedding service, a vector-store cache, an LLM client, and an external persistence/queue layer. The embedding model, vector store, LLM backend, similarity threshold, and cache-management policy are designed to be replaceable so that the project can evolve beyond the specific demo stack.

1.2 Assumptions and Design Constraints List

- The system handles text prompts only. Images, audio, and multimodal inputs are out of scope for v1.
- Two prompts are considered cache-equivalent when the distance between their embedding vectors is below a configurable threshold under a chosen similarity function.
- Each major component runs as a separate process and communicates over the network, even when all components run locally on one machine.
- The demo stack uses sentence-transformers for embeddings, FAISS for vector search, OpenAI API as the remote LLM backend, Go for the orchestrator, Python/FastAPI for ML-related services, Redis for persistence and asynchronous updates, and Docker/docker-compose for local deployment.
- The design must not depend on one specific embedding model, vector database, or LLM provider. These are hidden behind interfaces.
- The cache has a bounded capacity. When the cache is full, an admission/eviction policy decides which entry to admit and which entry to evict. The default policy is always admit plus LRU or LFU eviction.
- Cache state should survive process restarts. Redis stores replies and policy metadata; the vector index can be rebuilt from persisted entries at startup.
- The v1 user interface is a CLI. A simple web chat box may be added later on top of the same orchestrator API.
- Production-grade authentication, mTLS, rate limiting, and service-mesh security are not part of v1, but the architecture leaves room for them.

1.3 Distributed & Parallel Aspects

The project is inherently distributed because the main responsibilities have different resource needs and lifecycles. Embedding computation is ML-oriented and may benefit from a separate Python runtime or GPU-capable host; vector search is memory-oriented; the orchestrator is mostly I/O-bound and benefits from concurrency; Redis provides shared durable state and queueing; and the remote LLM is an external service accessed through a network API. Splitting these concerns into separate processes allows each component to be developed, deployed, replaced, and scaled independently.

Compared to a monolithic implementation, this design introduces network latency, serialization overhead, partial failures, retries, consistency issues between the vector index and persistent storage, and more operational complexity. The design addresses these difficulties by defining narrow REST contracts, making cache updates idempotent, using Redis as the external source of truth, sending slow cache-update work through an asynchronous queue, adding timeouts and clear error handling to remote calls, and packaging services with Docker so that local and cloud deployments use the same component boundaries.

The project is interesting as distributed software because it is a look-aside semantic cache in front of an expensive remote backend. A lookup itself has a cost: it requires embedding computation and vector search. The system is beneficial when that cost is much lower than calling the remote LLM, which is expected for repeated or semantically similar prompts. The distributed design also allows the cache layer to grow independently from the LLM backend and to support future replacements of the embedding model, vector store, or provider API.

1.4 Dependencies

Dependency	Use in project	
sentence-transformers	Generates embedding vectors for input prompts, demo model all-MiniLM-L6-v2.	
FAISS	In-memory vector similarity search over stored prompt embeddings.	
OpenAI API	Remote LLM backend for demo completions.	
Redis	Persistent store for replies, metadata, and asynchronous cache-update queue.	
Go 1.22+	Implements the cache orchestrator and LLM client.	
Python 3.11 + FastAPI	Implements embedding and vector-store services.	
Docker / docker-compose	Runs all components locally with production-like process boundaries.	

1.5 Issues List

- Tune the default similarity threshold for the selected embedding model using a representative prompt set.
- Decide whether long prompts should be rejected, truncated, chunked, or summarized before embedding.
- Decide whether FAISS index persistence should be done by rebuilding from Redis on startup or by storing the FAISS index file directly.
- Define exact retry behavior for LLM timeouts and provider errors.
- Choose the final metrics stack. Current candidate: Prometheus + Grafana.
- Decide whether the future web chat UI is part of the final submission or only a follow-up extension.

1.6 To-do List + Expected time-tables

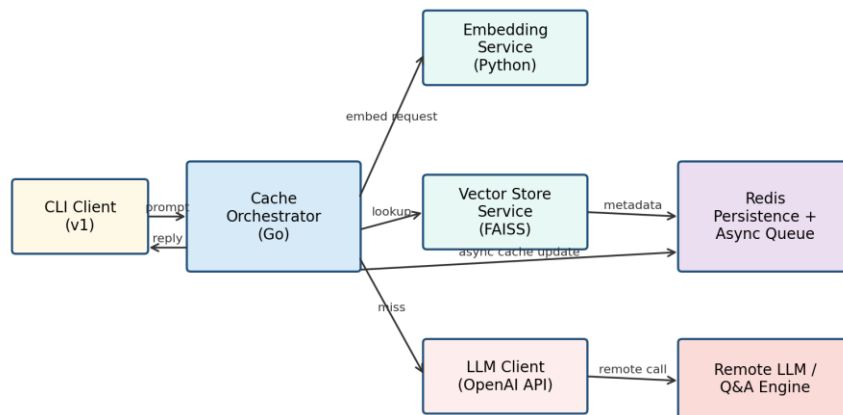
The project is planned for a three-person team over one academic semester, approximately fourteen weeks.

Week	Milestone
1-2	Initial project idea discussion, requirements gathering, and technology research.
3-4	Finalize system architecture (HLD), define REST APIs, choose technologies, and prepare repository structure.
5	Implement Embedding Service v1 using sentence-transformers.
6	Implement Vector Store service v1 using FAISS and similarity search functionality.
7	Implement Go orchestrator skeleton and basic communication between services.
8	Implement LLM Client integration, Redis persistence, and error handling mechanisms.
9	Implement caching policies (LRU/LFU) and asynchronous cache-update queue.
10	Implement CLI commands such as query, stats, flush, and policy switching.
11	Perform benchmarking, integration testing, and runtime validation.
12	Finalize deployment setup, prepare demo, complete documentation, and submit the project.

2 Logical Architecture

The application consists of a client-facing cache orchestrator and several replaceable backend services. The client sends a prompt to the orchestrator. The orchestrator asks the embedding service to convert the prompt into a vector, sends the vector to the vector-store service, and checks whether the closest cached vector is within the configured threshold. On a cache hit, the cached reply is returned. On a miss, the prompt is forwarded to the LLM backend. When the LLM reply arrives, the orchestrator returns it to the user and updates the cache asynchronously through Redis.

Logical Architecture - LLM Semantic Cache



2.1 CLI Client

Command-line interface for users. It sends prompts to the orchestrator and displays the reply, whether the response came from cache, the similarity distance, and latency. It also supports administrative commands such as stats, flush, and policy switching.

2.2 Cache Orchestrator

The central service and request coordinator. It receives prompts, calls the embedding service, queries the vector store, decides hit or miss, calls the LLM backend on miss, and schedules cache updates.

2.3 Embedding Service

A pluggable service that converts text prompts into embedding vectors. The demo uses sentence-transformers, but the orchestrator only depends on the REST interface.

2.4 Vector Store Service

Stores embedding vectors and finds the closest vector under a configurable similarity function and threshold. The demo uses FAISS, but the interface allows replacement by another vector database.

2.5 LLM Client / Remote LLM Backend

Encapsulates the remote LLM API. The demo calls OpenAI API, but the design supports other remote LLM providers through the same backend interface.

2.6 Persistence and Queue Layer

Redis stores cached replies, metadata, and policy counters. It also provides an asynchronous queue for cache updates so that slow persistence does not block the user response path.

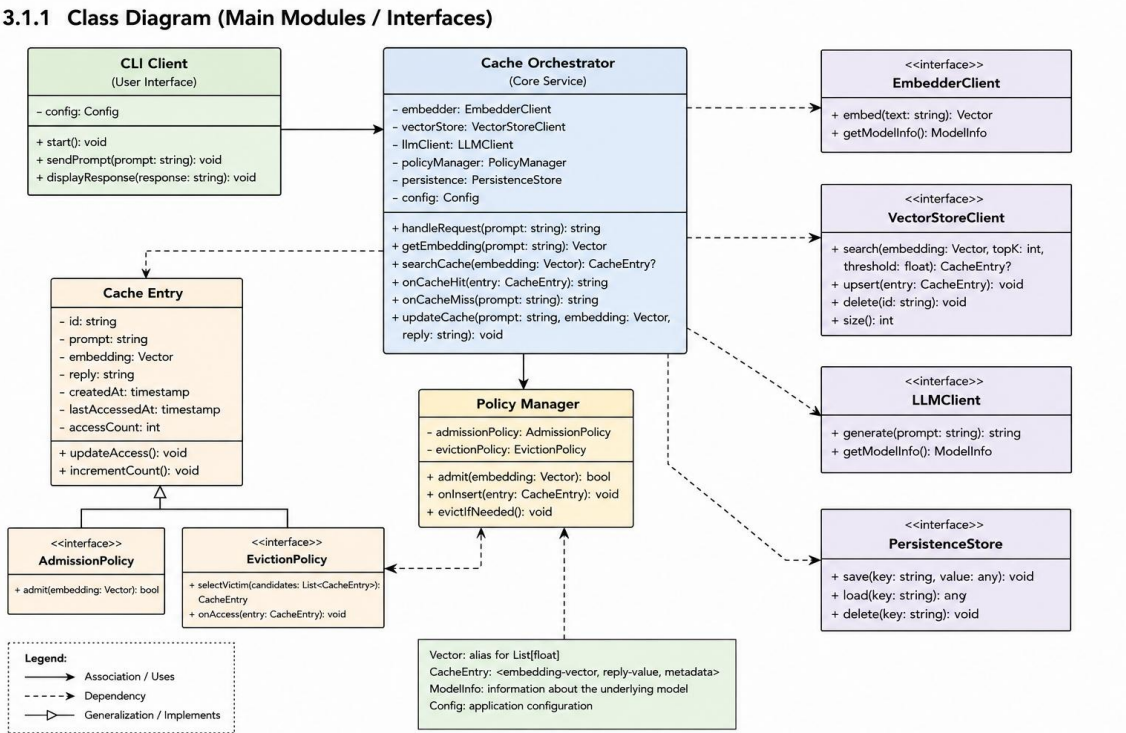
2.7 Policy Manager

Applies admission and eviction policies when the cache is full. The default design supports always-admit with LRU or LFU eviction, and future policies can be added through the same interface.

3 Design

3.1 Modules / Main Classes

3.1.1 Class Diagram

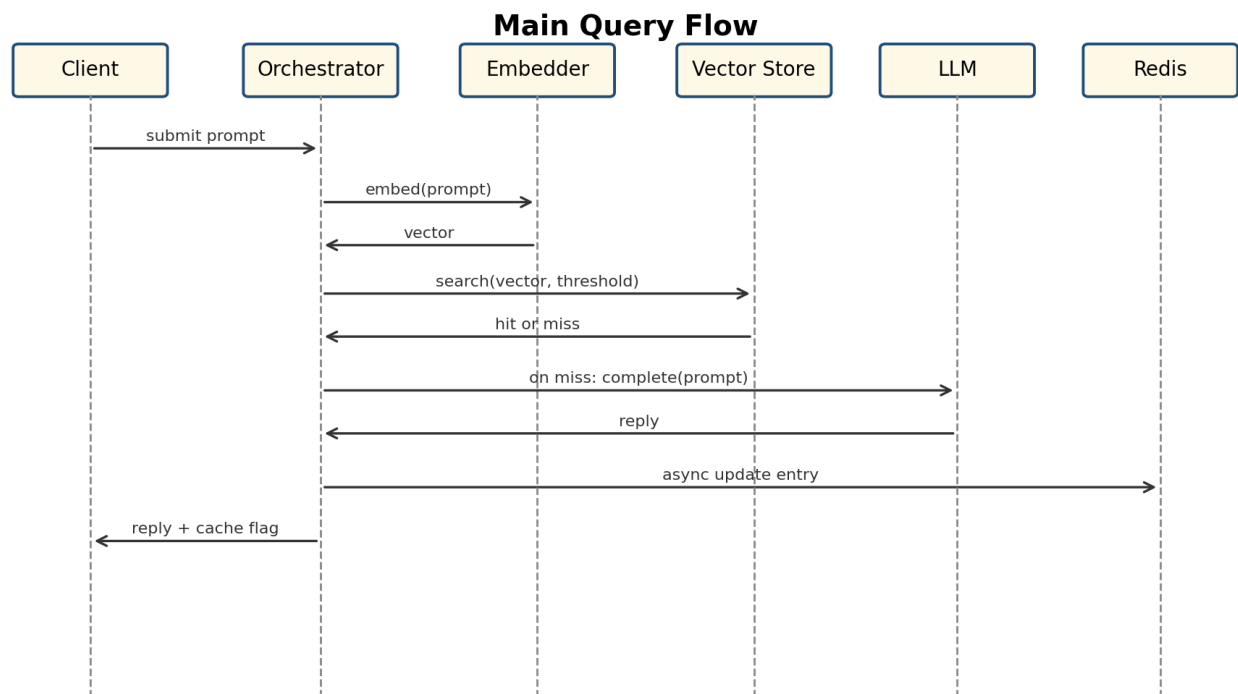


3.1.2 Class details

Module/Class	Responsibility	
CacheOrchestrator	Coordinates the query flow and owns hit/miss decision logic.	
EmbedderClient	Interface to embedding service.	
VectorStoreClient	Interface to vector store.	
LLMBackend	Interface to remote LLM provider.	
PolicyManager	Decides admission and eviction when capacity is reached.	
PersistenceStore	Stores replies, vectors metadata, policy counters, and stats.	
CLIClient	User-facing command-line tool.	

3.2 Flows

The main flow is the query flow. The important examples are cache hit, cache miss, and administrative operations.



Example 1 - Cache hit

- User submits: "Explain virtual memory."
- The orchestrator obtains an embedding vector for the prompt.
- The vector store finds a cached prompt such as "What is virtual memory?" with distance below the threshold.
- The orchestrator returns the cached reply and marks the result as `cache_hit=true`.
- Policy metadata is updated to record access frequency or recency.

Example 2 - Cache miss

- User submits a new prompt that has no close vector in the cache.
- The vector store returns no match below the threshold.
- The orchestrator forwards the prompt to the LLM backend.
- The LLM reply is returned to the client.
- The orchestrator enqueues an asynchronous cache-update event to store the new embedding-reply pair.

Example 3 - Cache full

- A new reply should be cached, but the configured capacity is full.
- The policy manager checks whether the new entry should be admitted.
- If admitted, the policy manager selects a victim using LRU or LFU.
- The victim is removed from Redis and the vector store, and the new entry is inserted.

3.3 User interface(s)

The v1 user interface is a command-line client. A user can submit a prompt and receive a reply, together with metadata such as cache hit/miss, distance to the matched prompt, and round-trip latency. The CLI also exposes simple administrative commands for statistics, cache flush, and policy selection. A future web chat box can be implemented on top of the same REST API without changing the backend design.

3.4 Setup

- Install Go 1.22+, Python 3.11+, Docker, and docker-compose.
- Create environment variables for the selected LLM provider, such as OPENAI_API_KEY.
- Build containers for orchestrator, embedding service, vector-store service, LLM client, Redis, and CLI client.
- Run docker-compose up to start all services locally.
- The orchestrator health endpoint verifies that embedding, vector store, Redis, and LLM client are reachable.
- For cloud deployment, the same containers can be deployed with Kubernetes manifests, with Redis configured as a managed or containerized service.

4 Use cases

Use case	Actor	Alternative / failure scenario
Submit prompt - cache hit	End user	If vector store is unavailable, system returns a clear service error.
Submit prompt - cache miss	End user	If remote LLM times out, system returns a meaningful timeout/error response.
Evict entry when full	System	If policy rejects admission, reply is returned to user but not stored.
View cache statistics	Developer/operator	If Redis is unavailable, stats command reports persistence unavailable.
Flush cache	Developer/operator	If partial failure occurs, the orchestrator reports which component failed.
Switch policy	Developer/operator	Invalid policy name is rejected without changing current policy.

5 References - external papers/packages

Reference / package	Used for
sentence-transformers documentation and all-MiniLM-L6-v2 model	Prompt embedding generation for the demo.
FAISS documentation	Efficient vector similarity search.
OpenAI API documentation	Remote LLM completion backend for demo.
Redis documentation	Persistence and asynchronous queue using Redis Streams.
Docker / docker-compose documentation	Local multi-process deployment.
REST/HTTP + JSON conventions	Inter-process communication contracts.